

XploreQuest

Product Requirements Document

1. Product Overview

XploreQuest is a mobile app (Android/iOS) designed specifically to manage and participate in (Explore Races / Explorace) events (a type of treasure hunt / adventure race held in open areas such as parks, lakes, and small towns in Malaysia). The app solves common problems observed in real events (e.g., Tasik Titiwangsa): large areas → difficulty finding checkpoints, overcrowded teams, wasted time getting lost, non-linear ordering, and manual registration.

Core Value Proposition

Make Explore Race events smoother, more fun, & fairer – easy organizer setup, participants (team leader only) use a simple app without information overload.

Target Users

- Organizer/Admin: Event organizers (universities, corporates, communities, government bodies).
- Crew/Marshal: Volunteers/staff at checkpoints (using the organizer side to verify).
- Participant (Team Leader): 1 person per team (2–6 members) – other members view progress from the leader's phone only.

Platform & Tech Stack

Cross-platform (Expo - React Native)

Google Maps SDK

Firebase (Auth, DB, Notifications)

2. Product Objectives

- Reduce participant frustration
- Prevent checkpoint overcrowding
- Automate registration & attendance
- Support small to large-scale events

MVP Success Metrics

- 80% teams complete race successfully
- Setup time < 30 minutes
- 100% automated check-in
- <1% crash rate

3. Key Features

Feature	Description
Event Registration & Join	Team leader downloads app → enters event code → inputs team name → requests to join → admin approves/rejects.
Attendance Check-In QR	Crew generates an attendance QR code (no expiry) → leader scans → system marks team as present. Absent teams are auto-excluded from the starting checkpoint draw.
Map & Boundary Setup	Admin draws the race area boundary → displayed visually on the participant's app.
Checkpoint System	Admin pins checkpoints, names them, and toggles visibility on the map.
Staggered / Cyclical Start	Auto-randomly assigns a starting checkpoint → cyclical order (e.g., start CP6 → 7 → ... → 5). Late arrivals: auto-assigned to less crowded checkpoints.
Checkpoint Verification QR	Crew selects team → generates a team-specific QR code → leader scans → checkpoint completed → next clue/map update unlocked.
Marking Location	Team leader long-presses/clicks map → pop-up to mark checkpoint (1-N) → restriction: cannot mark already-completed checkpoints, old marks auto-removed when completed.
Progress & Dashboard	Team leader views map with next checkpoint highlighted, completed list, and unlocked clues.

Push Notifications	Pre-race assignment, checkpoint completion, attendance reminder.
--------------------	--

4. Security Aspects

Layer 1: Authentication & access control

- RBAC (Role-Based Access Control) — Firebase custom claims to enforce that organizer, crew, and participant roles can only access their respective endpoints. A team leader should never be able to call a crew API endpoint.
- Brute-force protection — rate-limit failed join attempts using event codes. Without this, someone could enumerate valid event codes.

Layer 2: QR code & data integrity

- HMAC-signed QR payloads — instead of storing a static string, generate QR content as `{teamId}:{checkpointId}:{timestamp}:{HMAC-SHA256 signature}`. The server verifies the signature before accepting. This prevents forgery even if someone photographs another team's QR.
- One-time use flag — store a boolean scanned: true in Firebase Realtime DB. Once scanned, the same QR payload is rejected even if replayed. This directly counters the no-expiry weakness.
- Anti-replay window — even without expiry, you can reject QR payloads with timestamps older than the race duration (e.g., older than 6 hours).

Layer 3: Location & session security

- Mitigation: When a team scans a checkpoint QR, the app sends GPS coordinates to the server. The server independently checks if those coordinates fall within the boundary polygon stored in Firebase. If not, the scan is rejected — even if the QR itself is valid.
- Session tokens — each team gets a session JWT on race start. All scan requests must include this token. Prevents teams from sharing QR screenshots across events.

Layer 4: Infrastructure & data protection

- Input sanitization — team names, event codes, and any free-text fields should be validated both client-side and server-side to prevent injection into Firebase queries.
- No sensitive data on device — don't store event layout, all checkpoint locations, or other teams' progress in local storage. Pull only what's needed for the current task.

5. Final Year Project Security & CIA Triad Analysis

As a Cyber Security student project, mapping XploreQuest to the CIA Triad (Confidentiality, Integrity, and Availability) is essential. This transitions the application from a standard system-building project into a rigorous, research-oriented cybersecurity thesis. Below is an academic-grade analysis of the existing security controls and proposed security enhancements.

5.1 Confidentiality (C)

Confidentiality ensures that only authorized entities can access sensitive information (such as hidden checkpoint coordinates, event layouts, participant real-time GPS locations, and session credentials).

Existing Confidentiality Controls:

- Role-Based Access Control (RBAC): Enforced using Firebase Custom Claims to restrict APIs by role (Organizer vs. Crew vs. Participant).
- Zero-Trust Local Footprint: The mobile app does not store the full event layout, hidden checkpoints, or competitor positions on the local device.
- Brute-Force Protection: Rate-limiting failed join requests to prevent attackers from script-brute-forcing 6-character event codes.

Proposed Confidentiality Enhancements (For FYP):

- Ephemeral Location Logging (PDPA Compliance): Avoid permanent database storage of participant GPS coordinates. Keep coordinates strictly in memory during the race and purge them immediately post-event to comply with Malaysia's Personal Data Protection Act (PDPA) 2010.
- Hardware-Backed Secure Storage: Store JWT session tokens, API keys, and event secrets in the device's hardware-backed vault using Expo SecureStore (iOS Keychain / Android KeyStore) rather than unencrypted AsyncStorage.
- Field-Level Firestore Security Rules: Implement strict document-level rules enforcing write/read permissions based on authenticated user claims.

5.2 Integrity (I)

Integrity guarantees that data cannot be modified in an unauthorized or undetected manner. In XploreQuest, integrity prevents cheating, coordinate spoofing, scanning duplicate QR codes, or falsifying completions.

Existing Integrity Controls:

- HMAC-Signed QR Payloads: QR codes generated by crew marshals use the format {teamId}:{checkpointId}:{timestamp}:{HMAC-SHA256} to prevent payload tampering.
- One-Time Scans & Replay Windows: Stores scanned: true in the database and rejects scans with timestamps older than 6 hours to prevent QR screenshot sharing.
- GPS Geofencing: Verifies that the scanning device lies inside the admin-defined boundary polygon.

Proposed Integrity Enhancements (For FYP):

- Advanced GPS Spoofing Detection (Velocity Check): Mitigate mock-location spoofing apps by adding a Telemetry Velocity Check on the backend ($v = \Delta d / \Delta t$). If speed between sequential scans exceeds human physical limits (e.g., 100m/s), flag the check-in as fraudulent.
- Atomic Database Transactions: Use Firestore Transactions (runTransaction) to guarantee atomic increments of completed checkpoint states, preventing race conditions from simultaneous scanning attempts.

5.3 Availability (A)

Availability guarantees that systems and data are accessible to authorized users when needed. In adventure races, this represents resilience against poor network coverage in parks or forests, as well as server downtime.

Existing Availability Controls:

- Cloud Infrastructure: Utilizing highly available Firebase cloud databases and serverless architecture to auto-scale with participant demand.

Proposed Availability Enhancements (For FYP):

- Offline Maps & Resilient Sync: Implement local caching of maps (using Mapbox offline tiles) and allow the app to cache scan payloads locally and sync them back once internet service is recovered.
- Device Recovery Resilience: Store the active session states in the cloud so that if a team leader's phone battery dies, they can immediately log in from another team member's phone and resume the race without losing progress.
- DDoS Protection: Guard backend APIs and Cloud Functions with Firebase App Check to block unauthorized scripts or bot requests.

5.4 Comprehensive CIA Triad Checklist for Thesis

Triad Component	Asset / Threat	Current Control	Proposed FYP Enhancement
Confidentiality	Hidden Checkpoints	No local storage of layout on device	Enforce field-level Firestore read rules mapped to user claims.
Confidentiality	Credentials / JWT	Custom JWT Role Claims	Store JWT in Expo SecureStore (hardware KeyStore/Keychain).
Confidentiality	Participant Location	Strictly limited views	Implement ephemeral GPS logging with immediate post-race deletion.
Integrity	Checkpoint QR Forgery	HMAC-SHA256 signature	Integrate a Server Secret Rotation

		verification	Policy for HMAC keys.
Integrity	QR Screenshot Sharing	Replay-window & scanned flag	Add dynamic QR codes (rotating every 30s) or OTP-style QR payloads.
Integrity	Location Cheating / Spoofing	Boundary polygon verification	Add a GPS Telemetry Velocity Check (speed calculation) on the backend.
Availability	Live App Crashes	High-availability Firebase DB	Sync all local client states to cloud databases dynamically.
Availability	Spotty Internet Connection	Standard online-first Firebase	Offline map-tile caching and queued offline transactions.